PunkBackup — Troubleshooting Manual

> This manual shows the app in **English** — go to **"Settings"** → **"English"** if it isn't already switched (instant, no restart).

The app closes by itself shortly after opening

This is Avast (or a similar antivirus), not a bug in PunkBackup — see the "Important: this app is not code-signed" section of the Installation Manual for why, and the fix (add an exception for the PunkBackup install folder in Avast, restore it from Quarantine/Virus Chest if it landed there).

If a backup you started from your iPhone right after opening PunkBackup seems to have stopped for no reason, this is almost certainly why: Avast scans the app for its first ~10 seconds, then silently closes and reopens it — any backup run active at that exact moment gets cut off along with it (see the Installation Manual for the full explanation). Nothing already uploaded is lost — just start the backup again from your iPhone once the app is back open. Adding the Avast exception stops this from happening again.

"Start backup" does nothing / doesn't change state

- Fully close the app (not just minimize it) and reopen it from the Desktop icon.
- If it happens again, you should now get an error window with the exact detail (instead of failing silently) — send us that text.
- Make sure there isn't another copy of the app already open (check the taskbar).

The iPhone can't reach the server

1. Confirm the iPhone and the PC are on the **same WiFi network** (check the exact name in WiFi settings on both — some routers have separate 2.4GHz/5GHz networks with similar-looking names).
2. From Safari on the iPhone, try opening the address the app shows you (e.g. `http://PC-NAME.local:8787/health`). If it doesn't load, try the "Fallback IP" the app also shows.
3. Confirm the server is on in the PC app ("Status: Listening on port..."). It automatically retries a few times (briefly showing "Starting backup...") if the port doesn't free up right away — common right after antivirus software closes and reopens the app. If it still fails after those retries (e.g. the port is genuinely used by something else), the app shows a real error message explaining why instead of falsely claiming it's listening — if you see that error, close whatever else might be using port 8787 (or another copy of PunkBackup) and try again.

4. Check the Firewall rule exists: PowerShell (as administrator) → `Get-NetFirewallRule -DisplayName "PunkBackup"`. If it's missing (e.g. you skipped that checkbox during install), add it with `New-NetFirewallRule -DisplayName "PunkBackup" -Direction Inbound -Protocol TCP -LocalPort 8787 -Profile Private -Action Allow`.

401 error "Missing or invalid X-Backup-Token header"

- The token in the Shortcut doesn't match your profile's token on the PC.
- In the Shortcut, check **every** action with an `X-Backup-Token` header — the value must be the `Token` variable **chip** (colored background), never the literal word "Token" typed as plain text.
- Copy the token again from the app ("Copy token" button on the profile) and paste it into the corresponding **Text** action in the Shortcut.
- Since v1.7.14, the activity log records this too (once per device, not once per photo): "Backup request received from [IP] but no profile matches that token."

403 error "This profile is paused"

- The profile is **Paused**. Open the app → "Main" or "Settings" → switch it to **Active**.
- Since v1.7.14, the log also records this once: "Backup request received from [profile] but this profile isn't ready to receive it."

409 error "no destination folder configured"

- That profile has no destination folder. Go to **"Settings"** → "Choose folder..." for that profile.
- Since v1.7.14, the log also records this once, with the same message as the 403 error above (both mean "this profile isn't ready to receive a backup").

A profile's card says "USB not connected" (since v1.7.13)

- It means exactly that: that profile's destination folder points at a USB drive that isn't currently connected to the PC. This isn't a PunkBackup error — it's real information, and before v1.7.13 it wasn't shown clearly (the card used to say "0 files, never" as if the destination were simply empty, and the log said nothing when the Shortcut actually tried to reach it).
- Fix: plug in that specific USB and wait a few seconds — the card updates on its own (automatic refresh every ~1.5s), no need to restart the app or the profile.
- If the iPhone's Shortcut runs while the USB is disconnected, every attempt fails silently on the phone's side (this is a Shortcuts limitation, not PunkBackup's — see "The phone freezes while the Shortcut runs" below) while the app logs a line in the activity log like "Destination folder not reachable — is the USB drive connected?" — once per disconnect (since v1.7.19; it used to repeat every minute), so a Shortcut retrying hundreds of photos doesn't flood the log. It warns again if you reconnect the drive and it later disconnects again.

- A folder that doesn't exist YET but is on a drive that IS connected (e.g. a brand-new profile's very first backup) does NOT trigger this warning — that folder just gets created normally.

The Shortcut runs but doesn't upload any new photo

- Check **"Find Photos"** has no stray filter beyond the `Date Taken is before Limite` one (see the alternate Manual Build guide's step 3 for what this should look like) — a different filter sneaking in makes it return 0 results silently, with no visible error.
- Check **`Limit`** is turned **on** and set to **50** on `Find Photos` — this is mandatory (not optional): without it, `Find Photos` fails outright on a large library, even doing nothing but counting results.
- Confirm under **iPhone Settings** → **Privacy & Security** → **Photos** that the Shortcuts app has access ("All Photos" / "Always Allow") — with limited access, a photo you just took or saved may simply not be visible to the Shortcut yet, with no error shown.

All new photos are landing in the same folder (the current month)

This happens when the server receives an empty date for each photo, and `_year_month_dir()` falls back to "right now" — so everything ends up in the current month's folder regardless of when the photo was actually taken.

- **Almost always confirmed cause**: the chip inside the **Format Date** action (the one that builds the `TakenAt` variable — see step 4 of the alternate Manual Build guide) got silently reconfigured to point at a different attribute (e.g. "Name") instead of **Date Taken**. This usually happens with no visible error, right after adding or moving another action inside the same "Repeat" block — Shortcuts sometimes silently reconfigures neighboring chips.
- **How to confirm it**: inside the Shortcut, open section 4's "Format Date" action and tap its chip — it should say **Date Taken**. If it says anything else, that's the problem.
- **Fix**: re-select "Date Taken" on that chip. Photos uploaded AFTER this fix will land in their correct folder automatically — nothing else needs to change for new uploads.
- **Photos already uploaded to the wrong folder** (before the fix) don't reorganize themselves — they stay where they landed. If this happened to a large number of files, it's a one-off maintenance job (reading the real date directly from each file, or forcing a re-send from the phone), not something the Shortcut or server fixes automatically.

A video shows today's date in another app (PhotoPrism, Finder...), even though it's in the right Year/Month folder

This is expected and already fixed automatically — nothing you need to do. Some videos (mainly ones imported from other apps, not straight from the camera) need an automatic retry to upload correctly at all; that retry process used to leave the video's own internal date stamp set to the day it was retried, instead of the real recording date — even though the folder it landed in was always correct. The server now corrects that internal date automatically for every video it receives, and already went back and fixed every video backed up

before this correction existed. If you still see a wrong date in another app after this, that app may simply be reading a different field than the one being corrected — not something to chase further on the PunkBackup side.

My library is huge (thousands of photos) — will the full backup ever finish?

The Shortcut sweeps your library backward in bounded blocks of 50, newest photos first, advancing automatically block after block within one run — this is already built into the ready-made Shortcut file from the iPhone Setup Manual, no setup needed on your end. (If you built the Shortcut by hand instead, or just want to understand exactly how this works, see the "Set up the backward block-sweep" step of the alternate Manual Build guide.) This is what makes a large library actually finish, instead of either timing out (no `Limit`) or getting stuck re-checking the same fixed set forever (a plain `Limit` with no way to advance).

- The number of blocks per run is controlled by the `Repeticiones` variable (a `Text` action near the top, defaults to `50`). Confirmed working with `Repeticiones` up to 50 (≈2500 photos checked in one run). For your very first full backfill of a large library, it's fine to run the Shortcut several times in a row rather than pushing `Repeticiones` very high on the first try — watch the PC app's activity log (use "Expand" for a bigger view) to see it's keeping up before raising the number.
- **Videos could arrive empty (0 bytes), or Shortcuts could fail outright trying to read them (the "WFPhotoMediaContentItem produced no file representation..." alert) — both are now fixed (2026-10-23).** The server never records a video like that as a real backup, so before the fix the same video would simply retry itself on a later run. Root cause: Shortcuts sometimes can't read a video's real data straight from the Photos library (an iCloud asset not yet downloaded locally, or an uncommon codec/container — ProRes, HDR/Dolby Vision, Cinematic mode, Spatial Video). The Shortcut now runs every video through "Encode Media" (`Size: Passthrough`) before uploading it, instead of trying the raw item first — confirmed no real quality loss (same codec, resolution, bitrate via `ffprobe`). Photos are unaffected (they skip this step entirely). If you built the Shortcut by hand before this date, see section 4 (step c.0) of the alternate Manual Build guide to update your copy.
- If a run gets interrupted (you leave home, or tap Stop), nothing is lost — anything already uploaded stays backed up permanently. The next run just starts sweeping from your newest photos again rather than exactly where it left off. Re-checking those already-completed blocks never transfers any file again, but — see the "How fast is a backup, really?" section right below — it isn't necessarily fast either; budget real time for it, not zero.
- If the Shortcut itself appears to freeze with no error and no visible cause (rare, but a known Shortcuts app quirk unrelated to this system), force-quit it from the app switcher and run it again — nothing gets corrupted by an interrupted run, see the point above.

How fast is a backup, really? (measured in this environment)

These numbers come from real production uploads on this project's own setup — an **iPhone 15**, a **home WiFi network**, and a **Windows PC (Dell)** — derived directly from the server's own timestamps for every file it has ever received, refreshed using only the **most recent few days** of real uploads (about 1,340 files) so the numbers reflect current conditions rather than an average blended with the very first test runs.

File type	Share of a typical library	Avg. file size	Typical time per file
JPEG	~57%	0.24 MB	~1.7 s
HEIC	~30%	2.0 MB	~6.5 s
PNG	~5.5%	2.9 MB	~6.6 s
MP4	~3%	8.3 MB	~9.7 s
MOV	~4%	23.6 MB	~16.7 s

Most of the time for a small photo is **not** network transfer — it's the fixed overhead of the Shortcut's two requests per item (`/check`, then `/upload`). That overhead barely changes with file size, so it dominates small photos and matters less and less for bigger videos, where actual transfer speed becomes the main factor.

Projection by `Repeticiones` (the variable that controls how many 50-item blocks one Shortcut run sweeps — see the "Tuning `Repeticiones`" note in the iPhone Setup Manual), split out by file type using this same real mix, for a **first-time backup of brand-new files**:

Repeticiones	Total items	JPEG	HEIC	PNG	MP4	MOV	Time (new items only)
10	500	287	152	27	16	18	~35 min
20	1,000	574	303	55	31	37	~1 h 10 min
30	1,500	861	455	82	47	55	~1 h 45 min
40	2,000	1,148	607	109	63	73	~2 h 20 min
50	2,500	1,435	759	136	78	92	~2 h 56 min
75	3,750	2,152	1,138	205	118	137	~4 h 24 min
100	5,000	2,870	1,517	273	157	183	~5 h 52 min

Repeticiones	Total items	JPEG	HEIC	PNG	MP4	MOV	Time (new items only)
150	7,500	4,305	2,276	409	235	275	~8 h 48 min
180	9,000	5,166	2,731	491	283	330	~10 h 34 min
200	10,000	5,740	3,034	546	314	366	~11 h 44 min
250	12,500	7,175	3,793	682	392	458	~14 h 40 min

`Repeticiones` up to **50** (≈2,500 items) is confirmed reliable on a real device by this project's own original testing. **`Repeticiones = 180` (≈9,000 items) has since also been confirmed reliable on a real device** — the Shortcut itself doesn't fail or crash at that size. Values above 50 that haven't been separately confirmed are still shown here for reference, but treat them as unverified until tested.

> **Important, found while investigating a real backup that hadn't finished after several sessions**: the table above only models time for **new** files. It assumes every item in the sweep needs a real upload — but once part of your library is already backed up (from an earlier, interrupted run), most of each new block's 50 items are "already backed up, skip" instead. Measured directly from the activity log: a skip (`/check` comes back "already backed up," no file transfer at all) still took a **median of about 20 seconds** in real recent runs — not the near-instant round trip you'd expect for a plain HTTP check with nothing to transfer (sample size is still small, and it's noisy — anywhere from a few seconds to several minutes). **This means a run that's mostly re-checking already-backed-up content can take noticeably longer than the table above suggests**, not shorter — likely explains why a large backup that gets stopped and resumed repeatedly can take much longer in wall-clock time to finish than the "new items only" estimate implies. The exact cause isn't confirmed yet (candidates: per-action overhead in the Shortcuts app itself, or `Find Photos` re-scanning a large library on every block) — treat the skip-heavy case as **slower, not faster**, than a first-time backfill of the same item count until this is narrowed down further.

Other caveats: - Your own photo/video mix shifts these numbers — a video-heavy library takes noticeably longer per item than a photo-heavy one. - Your own WiFi signal strength and other devices competing for bandwidth at the same time will shift these numbers in either direction.

The phone freezes while the Shortcut runs

- Go to **iPhone Settings** → **Display & Brightness** → **Auto-Lock** → **Never** (temporarily, while running a large backup).
- Runs with thousands of photos can take several minutes — that's normal, not a freeze.
- If it truly stops responding: a forced restart of the iPhone is safe, nothing is lost (the server-side run is just left unfinished, harmlessly).

The Shortcut's final notification shows blank numbers (New/Already had/Conflicts)

This never affects the actual backup — files keep uploading and saving correctly even when the notification comes back blank. If it happens, check the **"Get Contents of URL"** action pointing at `/run/finish` (see step 5 of the alternate Manual Build guide):

- The `X-Backup-Token` header must be under **Headers**, not inside **Request Body** → **Form**.
- **Request Body** → **Form** needs a `run_id` field set to the `RunID` variable — without it, the server rejects the request and the rest of that section (reading the counters, building the message) gets silently skipped, leaving the summary blank.

To confirm something was really saved while you check, look at the destination folder directly (see next section).

How to check files are really being saved

No technical steps needed: open the destination folder you chose in Windows Explorer — you should see Year/Month subfolders with your photos and videos inside.

Second copy (mirror) — messages and what they mean

See the Usage Manual's "Second copy (optional)" section for how this feature normally works. These are the messages you might see:

- **"Can't use that folder"** — you tried to set the second copy to the exact same folder as your main destination. Choose a different one.
- **"Second copy: [path] (not connected)"**, sync button greyed out — normal, expected state whenever that drive isn't plugged in right now. Not an error; plug it back in and it'll be ready to sync again.
- **"The second copy doesn't have enough space for everything pending"** — a heads-up shown before syncing, not something that stops it: it still copies what fits, starting with your newest photos, and picks up the rest once you free up space or swap in a bigger drive.
- **"The sync stopped before finishing ([error])"** — something interrupted a file mid-copy, usually either the drive filling up for real or getting unplugged instead of safely ejected. Nothing is lost — whatever copied successfully before that stays valid — just check the drive and tap "Sync now" again; it resumes exactly where it stopped.

- **"Couldn't complete the second-copy sync: [error]"** — a real, one-off failure (rare). The activity log (▼ Show activity) has the exact error text right after this line in the log.
- Reconnecting a second-copy drive that already has some files on it never re-copies what's already there, and never restarts the visible counter from zero — the running count you see already reflects everything on that drive, old and new combined.
- **"N failed"**, next to the counter during a sync — shows up when some files fail verification, almost always because something else is using the same USB drive at the same time (e.g. a real backup from the iPhone running in parallel). The main counter (**X / Y files**) only counts copies that actually got confirmed correct — it never counts a failed attempt as progress, so if you see it climbing very slowly alongside a growing failed count, that's a real sign something else is competing for that drive — not an app bug. Stop whatever else is using that same drive and sync again.

How fast is the second copy (mirror) sync?

Same real-measurement approach as the WiFi upload speeds above, but for the local USB-to-USB copy instead — measured from a real sync run in this project's own environment (607 real files, 3.11 GB, copied and verified).

File type	Count	Avg. size	Typical time	Approx. speed
JPEG	349	0.21 MB	~1.35 s	~0.15 MB/s
HEIC	117	2.23 MB	~2.24 s	~0.99 MB/s
PNG	57	1.75 MB	~1.70 s	~1.03 MB/s
MOV	82	32.62 MB	~8.27 s	~3.94 MB/s

The local copy is generally faster than a WiFi upload of the same file — especially for videos (~8 s locally vs. ~17-19 s over WiFi) — but for small JPEGs there's barely any difference (~1.35 s locally vs. ~1.7-2 s over WiFi). Same underlying reason as the WiFi numbers: most of the time for a small file is fixed per-file overhead (opening it, copying it, reading it back to verify, recording it), not actual data transfer — transfer speed only starts to matter once the file itself is big enough (videos).

Important caveat: this was measured on a small (4 GB), nearly-full USB stick — not a fresh, spacious drive. A flash drive with little free space left can behave worse than these numbers under real conditions (this test run itself ended by genuinely running out of space, with a noticeably higher rate of failed-verification copies right at the end — 58 out of 289, compared to 7 the first time the same drive was tested with more room to spare). Expect a healthy drive with real free space to do at least this well, likely better — but don't expect a nearly-full drive to keep up.

Still not working

Contact whoever helped you install the system, or check the project's GitHub repository to report the issue.